Chapter 46: JIT (Just-in-Time Compilation)

1. Overview

Angular templates are not directly interpreted by the browser. They are compiled into JavaScript — specifically into a chain of Ivy instructions (or, pre-Ivy, into `ViewFactory` / `NgModuleFactory` code) that the runtime executes to create DOM, bind data, and wire up change detection. That compilation has to happen *somewhere*, at *some point in time*. There are exactly two moments where it can happen:

- **Build time**, on a developer's or CI machine, before the app is ever shipped — this is **Ahead-of-Time (AOT) compilation**.
- **Run time**, inside the user's browser, right as the application boots — this is **Just-in-Time (JIT) compilation**.

For most of Angular's life (Angular 2 through Angular 8), JIT was the *default* for `ng serve` (development builds), while AOT was opt-in for production builds via `--prod` or `--aot`. Angular 9 (2020, with the Ivy renderer) flipped the default: AOT became the default for both `ng build` and `ng serve`. Angular later fully removed the ability to run a production app compiled with the runtime JIT compiler pipeline as a first-class mode, and modern Angular (v13+ with `ngcc` removed, and later fully deprecating `platformBrowserDynamic` compiler bootstrapping in favor of `bootstrapApplication`) treats JIT as a legacy/edge-case capability rather than a supported deployment strategy.

This chapter covers: what JIT actually did mechanically, why it existed and was convenient for development, why it was deprecated/removed as a shipped-to-production default, what `@angular/compiler` was and why shipping it to the browser was expensive and risky, and the narrow scenarios where runtime template compilation still matters today (e.g., compiling a component from a string at runtime — rare, but it exists via APIs like Ivy's runtime compilation utilities).

2. Core Concepts

2.1 What "compilation" means for an Angular template

Given a template like:

```
<div>{{ name }}</div>
<button (click)="onClick()">Go</button>
```

The Angular compiler must turn this into executable instructions — something conceptually like:

```
function MyComponent_Template(rf, ctx) {
  if (rf & 1) {
    elementStart(0, 'div');
    text(1);
    elementEnd();
    elementStart(2, 'button');
    listener('click', function() { return ctx.onClick(); });
    text(3, 'Go');
    elementEnd();
  }
```

```
  if (rf & 2) {
    textInterpolate(ctx.name);
  }
}
```

This translation — parsing HTML-like template syntax, resolving expressions, building an AST, and emitting instruction code — is what "the compiler" does. It's a genuinely large piece of software: an HTML/template parser, an expression parser, a template type-checker (in AOT), a static analyzer for decorators (`@Component`, `@NgModule`, `@Directive`, etc.), and a code emitter.

2.2 JIT: doing this in the browser, at bootstrap

In the JIT model, the build step (webpack + `tsc`) compiles your TypeScript classes and decorators as plain JavaScript classes with decorator metadata attached, but **does not** pre-compile templates into instruction code. Instead:

1. The bundle includes your component classes, their decorator metadata (component's `template`/`templateUrl` string, `styles`, `selector`, etc.), and the entire `@angular/compiler` package.
2. At runtime, when `platformBrowserDynamic().bootstrapModule(AppModule)` runs, Angular's JIT compiler:
    - Reads the `@Component`/`@NgModule` metadata reflectively.
    - Parses the template string into an AST.
    - Resolves directive/pipe matching, i18n, bindings.
    - Emits Ivy instruction functions **in memory**, as actual JS functions (historically via `new Function(...)`-style dynamic code generation, or Ivy's `jitExpression`/patching of `ɵcmp` definitions).
    - Patches the component class with the compiled `ɵcmp` definition so it can be instantiated.
3. Only after this whole pipeline runs for the root module and its transitive dependency graph does Angular render anything to the screen.

This is why old versions of `ng serve` (JIT dev builds) had a visible startup lag and why the dev bundle was so much larger than the prod (AOT) bundle — you were shipping a compiler, not just an app.

2.3 AOT: doing this at build time

With AOT, the Angular compiler (`ngc`, then later the Ivy AOT compiler integrated into the CLI/webpack pipeline) runs **during the build**, on the developer's machine or CI:

- It parses templates and emits Ivy instruction functions as **actual `.js` source code**, compiled alongside your TS.
- The `ɵcmp` static property is already fully populated at build time — no reflection over decorator metadata is needed at runtime, and in fact decorator metadata (like the raw `template` string) can be stripped from the shipped bundle entirely.
- `@angular/compiler` itself is **not** included in the production bundle — only `@angular/compiler-cli` was needed, and only at build time, on a machine that then discards it.
- The browser receives only compiled instruction code and runs it directly — no parsing, no code generation, no `eval`-like step at startup.

2.4 Why JIT existed at all

JIT wasn't an oversight — it solved real problems in Angular 2–8:

- **Fast dev iteration.** Early AOT builds were slow (whole-program recompilation on every template edit). JIT let `ng serve` pick up a template change and re-render without invoking the full AOT pipeline — useful before Ivy's incremental compilation made AOT fast enough for dev too.
- **Simplicity for library/demo code.** You could write a Angular app with no build step at all (template strings inline, load via `<script>` tags referencing UMD bundles + `@angular/compiler`, no webpack) purely by relying on the runtime compiler. This was genuinely used in early tutorials, Plunker/JSFiddle demos, and some legacy enterprise setups that didn't want a full build pipeline.
- **Runtime-determined templates.** If a template string legitimately wasn't known until runtime (e.g., a CMS delivering component markup dynamically, plugin-style architectures), JIT was the only way to compile it — you couldn't AOT-compile a template that doesn't exist at build time.

2.5 Why JIT was deprecated / removed as the default

Several forces converged against JIT-as-default:

1. **Bundle size.** `@angular/compiler` is a large, non-trivial payload (historically tens to 100+ KB gzipped on top of `@angular/core` / `common`). Shipping a full HTML/expression parser and code generator to every user, for every page load, purely so the app could compile *itself*, was pure waste — the compiler's job is done once and is identical for every user on every load (same templates, same output).
2. **Startup performance.** Parsing and code-generating templates in the browser, on every app boot, on every device (including low-power mobile devices), meant real, measurable delay before first render (`bootstrapModule` had to finish the entire JIT pipeline before the app tree could be created). AOT moved this cost to build time, paid once by the CI machine, never by the user.
3. **No template type-checking until runtime.** In JIT mode, a broken binding (`{{ user.nam }}` — typo) was only discovered when a real user's browser tried to render that template and threw at runtime. AOT's compiler runs at build time and can be configured (`strictTemplates`) to catch such errors as **build failures**, shifting an entire class of bugs left from "customer sees a broken page" to "CI fails, PR blocked."
4. **Security: shipping a code-generation capability to the browser.** The JIT compiler builds and evaluates JavaScript from string input (template strings, expression strings) inside the browser's execution context. Any pathway where an attacker could influence template content that reaches the JIT compiler (e.g., improperly sanitized dynamic template content, a compromised CDN delivering altered metadata) becomes a code-injection vector — conceptually adjacent to `eval()` of untrusted strings. Angular's own CSP (Content Security Policy) guidance flags JIT/ `new Function` usage as something that requires `unsafe-eval`, which is a CSP directive security teams specifically want to avoid granting. AOT-compiled apps need no `unsafe-eval` at all — the browser is only ever running static, pre-emitted, reviewed JS.
5. **Ivy made AOT fast and universal.** Before Ivy, AOT compilation was seen as heavyweight, produced larger and more indirect code (via factories/ `NgFactory` files), and had rough edges (e.g., issues with certain dynamic component patterns, `ModuleWithProviders` without generics, metadata restrictions). Ivy's locality principle (each component compiles independently, without needing whole-program knowledge) made AOT fast enough to use even in `ng serve` dev-mode rebuilds, removing JIT's last practical advantage (fast dev loop).

6. **CLI defaults flipped.** Angular 9 made AOT the default for *all* build types. Later Angular versions (v13+) removed View Engine entirely (Ivy-only), and the JIT compiler path for `@Component`s using `templateUrl`s and separate metadata resolution became increasingly vestigial, kept mostly for **unit testing** (`TestBed` still does JIT-style recompilation of test-time overridden templates, in a Node/JSDOM context, not the shipped app) and for a narrow set of dynamic-template runtime scenarios.

2.6 What remains of JIT today

- `TestBed` **/ Angular testing.** `TestBed.overrideComponent(...)`, `TestBed.configureTestingModule(...)` still perform JIT-style recompilation, because tests routinely need to substitute a template or mock a dependency at runtime, and paying the AOT cost per test run isn't practical. This is why test bundles still pull in `@angular/platform-browser-dynamic` / compiler-adjacent code, but this never ships to production users.
- **Runtime component creation from a string template — a rare, explicit escape hatch.** If an application must genuinely compile a template unknown until runtime (e.g., building a low-code/CMS-driven renderer), Angular exposes low-level APIs to do this, but it requires explicitly importing `@angular/compiler` and opting in — it is not automatic, and it is documented as a niche/advanced capability, not a supported everyday pattern. `Compiler` / `JitCompilerFactory` and, in older Angular, `ComponentFactoryResolver` combined with `@angular/compiler` were the mechanism; teams needing this today usually prefer safer alternatives (server-rendering the dynamic content, using `NgComponentOutlet` with a **pre-compiled** set of known components selected dynamically, or a sandboxed micro-frontend approach) precisely to avoid the bundle-size and security costs described above.
- `@angular/compiler` **as a build-time-only dependency.** It still exists in `node_modules` and is still essential — `@angular/compiler-cli` depends on it — but it runs on the build machine, not in the browser, for every standard Angular CLI application today.

3. Code Examples

```
// ── HISTORICAL JIT BOOTSTRAP (Angular 2–8 style, pre-Ivy-default) ──────────

// main.ts (JIT — the "dynamic" platform pulls in @angular/compiler)
import { platformBrowserDynamic } from '@angular/platform-browser-dynamic';
import { AppModule } from './app/app.module';

// "Dynamic" here literally means "compiles templates dynamically, at runtime,
// in the browser." This bundle includes @angular/compiler.
platformBrowserDynamic().bootstrapModule(AppModule)
  .catch(err => console.error(err));

// @Component metadata is just a string reference — the compiler resolves
// and compiles this template IN THE BROWSER, the first time the app boots.
import { Component, NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';

@Component({
  selector: 'app-root',
  templateUrl: './app.component.html', // fetched/inlined, then JIT-compiled at runtime
  styleUrls: ['./app.component.css'],
```

```
})
export class AppComponent {
  name = 'World';
  onClick() { console.log('clicked'); }
}

@NgModule({
  declarations: [AppComponent],
  imports: [BrowserModule],
  bootstrap: [AppComponent],
})
export class AppModule {}


// ── AOT BOOTSTRAP (Angular 9+ default; also today's standalone bootstrap) ──

// main.ts (AOT — the "static" platform; NO @angular/compiler shipped)
import { platformBrowser } from '@angular/platform-browser';
import { AppModuleNgFactory } from './app/app.module.ngfactory'; // pre-Ivy AOT artifact
// or, with Ivy AOT (Angular 9+), simply:
import { AppModule } from './app/app.module';

platformBrowser().bootstrapModuleFactory(AppModuleNgFactory) // classic AOT (pre-Ivy)
  .catch(err => console.error(err));

// Ivy AOT (Angular 9+): same bootstrapModule call, but the CLI build step has
// ALREADY invoked the AOT compiler and patched AppComponent.ɵcmp with fully
// compiled instruction code before this file is ever bundled for the browser.
platformBrowser().bootstrapModule(AppModule)
  .catch(err => console.error(err));


// ── MODERN STANDALONE BOOTSTRAP (Angular 14+, AOT-only, no NgModule) ──────

import { bootstrapApplication } from '@angular/platform-browser';
import { AppComponent } from './app/app.component';

// bootstrapApplication has NO "dynamic" counterpart in modern Angular —
// there is no supported "bootstrapApplicationDynamic". AOT is the only path.
bootstrapApplication(AppComponent).catch(err => console.error(err));


// ── THE RARE REMAINING RUNTIME-COMPILATION ESCAPE HATCH ────────────────────
// Compiling a component definition from a template string not known until
// runtime. Requires explicitly pulling in @angular/compiler yourself.

import '@angular/compiler'; // opt-in: pulls the compiler into the bundle
import { Component, Compiler, Injector, NgModule, ViewContainerRef } from '@angular/core';

@Component({ selector: 'dynamic-host', template: '' })
export class DynamicHostComponent {
  constructor(
    private compiler: Compiler,
```

```
    private injector: Injector,
    private vcr: ViewContainerRef,
  ) {}

  renderFromString(templateSource: string) {
    // Build a throwaway component class + module around the runtime string,
    // then JIT-compile it. This is exactly the mechanism removed from the
    // default pipeline — used here deliberately, and rarely, for CMS-driven
    // or low-code rendering scenarios.
    @Component({ selector: 'runtime-cmp', template: templateSource })
    class RuntimeComponent {}

    @NgModule({ declarations: [RuntimeComponent] })
    class RuntimeModule {}

    const moduleFactory = this.compiler.compileModuleSync(RuntimeModule);
    const moduleRef = moduleFactory.create(this.injector);
    const factory = moduleRef.componentFactoryResolver
      .resolveComponentFactory(RuntimeComponent);
    this.vcr.clear();
    this.vcr.createComponent(factory);
  }
}
```

4. Internal Working

JIT bootstrap sequence, mechanically:

1. `platformBrowserDynamic()` sets up a platform injector configured with the JIT `CompilerFactory` (as opposed to `platformBrowser()`, which assumes templates are already compiled).
2. `bootstrapModule(AppModule)` resolves the root module's metadata via `Reflect`/TypeScript's emitted decorator metadata (`__decorate`, `__metadata`) — this is why JIT mode needed the `reflect-metadata` polyfill loaded in `polyfills.ts` historically.
3. The `Compiler` service walks the `@NgModule`'s `declarations`, and for each component:
   - Reads `template`/`templateUrl` and `styles`/`styleUrls` off the decorator metadata object.
   - If `templateUrl` was used, this had to already be resolved to an inline string by the build step (webpack's `raw-loader`/Angular's template-url-inlining) — the *fetching* of the file happens at build time even in JIT mode, but the *parsing/compiling* of its contents happens at runtime.
   - Parses the template string with Angular's HTML parser into an HTML AST.
   - Parses embedded expressions (`{{ }}`, `(click)="…"`, `[prop]="…"`) with the expression parser into an expression AST.
   - Resolves which directives/pipes match which elements, using the module's compilation scope.
   - Emits Ivy instruction functions. In JIT mode, this emission is not writing `.js` files — it constructs the instruction closures directly as in-memory JavaScript functions (effectively dynamic function construction equivalent to `new Function(...)`), then assigns them onto the component class's `ecmp` (`ComponentDef`) property, monkey-patching the class after the fact.
4. Only once the root component's (and its entire dependency graph's) `ecmp` definitions have been

synthesized this way does Angular proceed to create the root `LView`, run the first change detection pass, and produce visible DOM.

5. Every subsequent page load, for every user, repeats this entire parse-and-codegen sequence from scratch — nothing is cached across sessions (aside from ordinary HTTP caching of the static JS bundle itself, which still contains raw template strings + the compiler, not compiled output).

Why this doubled bundle size: the shipped JS had to contain (a) the entire `@angular/compiler` package — HTML lexer/parser, expression lexer/parser, template AST, i18n extraction logic, output AST + code-emission logic, schema validation — plus (b) all the raw template/style strings for every component (uncompiled source, not compact instruction bytecode), plus (c) `reflect-metadata` and full decorator metadata retained on every class (normally erasable in AOT). Historical measurements from the Angular team cited JIT bundles as roughly on the order of 50-100%+ larger than equivalent AOT bundles for the same app, with the compiler itself being a significant fraction of that delta.

Why this delayed first render: compiling is CPU-bound synchronous work that runs on the main thread before any component tree exists. AOT apps skip straight from "parse JS bundle" to "instantiate root component and run change detection" — there is no intermediate "generate the code that will let us instantiate the root component" phase, because that code was already generated at build time.

5. Edge Cases & Gotchas

- **Template errors surfaced only at runtime, and only for the specific user whose code path hit them.** A conditionally-rendered template branch (behind an `*ngIf` that's rarely true) with a broken binding would compile "successfully" (JIT doesn't proactively compile every possible branch/component until it's actually instantiated in some paths, and even when it does compile it, JIT by default did not run Angular's strict template type-checker) and only throw when a real user's session actually rendered that branch — turning a should-be-compile-time bug into a production incident discovered via error monitoring, if at all.
- **No `strictTemplates`-equivalent enforcement in JIT.** AOT's template type-checker cross-references your `.ts` component class's typed properties against binding expressions in the template (e.g., catching `ctx.usre` typos, wrong argument counts to pipes, type mismatches in `[prop]="expr"`). JIT compilation, being a runtime process without a build-time TypeScript program to consult, historically skipped this depth of checking — it would only catch structural template errors (malformed HTML, unknown elements without `CUSTOM_ELEMENTS_SCHEMA`), not type errors.
- `unsafe-eval` **CSP requirement.** JIT's dynamic function construction is functionally equivalent to `eval` from a Content-Security-Policy enforcement perspective. Any app relying on JIT compilation (or the modern runtime-compilation escape hatch shown above) cannot deploy under a CSP that disallows `unsafe-eval` — a real blocker in security-sensitive environments (browser extensions with Manifest V3, some enterprise/government deployments, some Electron apps) that categorically forbid `unsafe-eval`. This is one of the most concrete, practical reasons teams needed to guarantee AOT-only compilation, not just prefer it.
- `reflect-metadata` **load-order footgun.** JIT mode required `import 'reflect-metadata'` (or later, Angular's own decorator metadata handling) to be loaded before *any* decorated class was evaluated; getting import order wrong in a JIT app produced cryptic "Cannot read property of undefined" errors during bootstrap that had nothing to do with your actual code.
- **Dynamic-template escape hatch reintroduces exactly the problems JIT was removed for.** If a team pulls `@angular/compiler` back in for CMS-style dynamic templates, they silently re-inherit the bundle-

size hit, the CSP `unsafe-eval` requirement, and the "errors only at runtime" problem for that portion of the app — teams should treat this as a deliberate, scoped, security-reviewed exception, not a casual convenience.

- `TestBed` **JIT vs. app AOT can behave subtly differently.** Because unit tests recompile components via `TestBed`'s JIT-ish pipeline (even for apps that are otherwise AOT-only in production), it's possible — though rare with Ivy — for a component to behave correctly under `ng test` but reveal an AOT-only quirk (e.g., certain metadata restrictions, or issues around `templateUrl` resolution paths) only when actually built with `ng build`. This is a real reason teams also run a production build (and ideally e2e tests against it) in CI, not just unit tests.
- **Confusing "JIT the compiler mode" with "just-in-time" in the general CS sense.** Interview candidates sometimes conflate Angular's JIT (runtime *template* compilation to Ivy instructions) with V8's JIT (runtime *machine-code* compilation of already-valid JavaScript). Angular's JIT/AOT distinction is entirely about *when Angular's own template compiler runs*; it has nothing to do with V8 optimizing the resulting JavaScript, which happens regardless of AOT or JIT and is outside Angular's control either way.

6. Interview Questions & Answers

Q1. What is the difference between JIT and AOT compilation in Angular?
JIT (Just-in-Time) compiles component templates into executable instruction code inside the user's browser, at application bootstrap time. AOT (Ahead-of-Time) performs that same compilation during the build (`ng build`), producing a bundle that already contains compiled instruction code, so the browser never needs to parse templates or generate code at runtime.

Q2. Why did Angular support JIT in the first place?
Mainly for fast development iteration (before Ivy made AOT itself fast) and for scenarios needing zero build tooling (inline template strings, script-tag-only demos). It also enabled compiling templates whose content genuinely isn't known until runtime, such as CMS-delivered markup.

Q3. Why was JIT deprecated as the default/production mode?
Interviewer intent: checks whether the candidate understands this was a deliberate, multi-factor engineering tradeoff, not an arbitrary version bump.
Four converging reasons: (1) bundle size — shipping the entire `@angular/compiler` package to every user is dead weight since compilation output is identical for everyone; (2) startup performance — parsing/codegen on the main thread before first render adds real latency, especially on mobile; (3) lost build-time safety — JIT errors surface only when a real user hits that code path at runtime, instead of failing CI; (4) security — runtime code generation from string templates is `eval`-adjacent and requires relaxing CSP (`unsafe-eval`), which security teams want to avoid. Ivy's fast, local, per-component AOT compilation removed JIT's last advantage (slow AOT dev loop), so Angular 9 flipped the CLI default to AOT everywhere.

Q4. What was `@angular/compiler` and why is it different from `@angular/compiler-cli`?
`@angular/compiler` is the actual compiler engine — the template/expression parsers, AST, and code emitter. `@angular/compiler-cli` is the build-tool wrapper (`ngc` and the Angular CLI's build integration) that *uses* `@angular/compiler` at build time, running template type-checking and emitting compiled output as part of `ng build`. In an AOT app, `@angular/compiler-cli` runs on the build machine and is never shipped; `@angular/compiler` itself was only shipped to the browser in JIT mode.

Q5. What concretely made shipping the compiler to the browser a "security liability"?
Interviewer intent: tests whether the candidate can name the mechanism, not just repeat "it's insecure."
JIT compilation constructs and executes JavaScript from string input (component templates/expressions) inside the browser's JS runtime — mechanically similar to `eval()`. This requires the page's Content-Security-Policy to permit `unsafe-eval`, which is exactly the capability CSP is designed to restrict, because it's the same capability an XSS payload would need to escalate from injecting markup to executing arbitrary logic. An AOT-compiled app ships only static, pre-reviewed instruction code and needs no such CSP relaxation, which is a meaningfully smaller attack surface, not just a stylistic preference.

Q6. Could a broken template ever pass in JIT mode but fail in AOT mode, or vice versa? Why?
Yes, primarily one direction: something that "worked" under JIT (a typo'd binding on a rarely-hit `*ngIf` branch, a subtly wrong pipe argument) could fail to even build under AOT, because AOT's template type-checker (`strictTemplates`) statically analyzes every declared template against the component's TypeScript types at build time, regardless of whether that branch is ever exercised at runtime. JIT, lacking a build-time TS program to consult, historically performed shallower checking and only failed when a user's session actually executed the broken path.

Q7. What is `platformBrowserDynamic` vs `platformBrowser`, and does either still make sense to use today?
`platformBrowserDynamic` bootstraps with the JIT compiler wired in (historically the default for `ng serve`); `platformBrowser` expects an already-compiled module/factory (the AOT path). Since Angular 9, the CLI's build pipeline compiles ahead-of-time regardless of which bootstrap call is present in source, and modern standalone bootstrapping (`bootstrapApplication`) has no "dynamic" counterpart at all — so in practice, for CLI-built apps, the distinction is now mostly historical; both effectively end up running against AOT-compiled definitions once processed by the build.

Q8. Are there legitimate reasons to still use runtime/JIT-style compilation today? Give an example.
Interviewer intent: separates candidates who think JIT is "just gone" from those who understand it's a deliberately narrowed, opt-in capability.
Yes, narrowly: when a template's content is genuinely unknown until runtime and can't be enumerated at build time — e.g., a low-code platform or CMS where end-users author component markup that must be rendered dynamically. Angular still exposes `Compiler.compileModuleSync` / `compileModuleAsync` for this, but it requires explicitly importing `@angular/compiler` and reintroduces the bundle-size and CSP costs JIT-as-default was removed for, so it should be a deliberate, scoped decision — many teams instead prefer `NgComponentOutlet` with a known, pre-compiled set of components selected dynamically by identifier, or server-side rendering of the dynamic content, to avoid pulling the compiler into the client bundle at all.

Q9. Why does (or did) `TestBed` still perform something like JIT compilation, even in an otherwise AOT-only app?
Tests routinely need to override a component's template or mock out dependencies (`TestBed.overrideComponent`, `configureTestingModule`) at run time, per test, and it isn't practical to run a full AOT build for every test permutation. So Angular's testing infrastructure recompiles affected components in the test (Node/JSDOM) environment using JIT-style mechanics — this never ships to production users; it's purely a test-time convenience, running once per `ng test` invocation, not once per end-user session.

Q10. How did Ivy change the calculus between JIT and AOT compared to View Engine (pre-Ivy)?
Interviewer intent: checks whether the candidate connects the Ivy rewrite to the JIT deprecation, rather than treating them as unrelated facts.
Pre-Ivy (View Engine), AOT compilation required whole-program knowledge to generate `NgFactory` files, making it comparatively slow and rigid, which is exactly why JIT remained the default for dev builds —

recompiling per-component-in-isolation wasn't possible. Ivy introduced the "locality principle": each component's instruction code can be compiled independently, without knowledge of the rest of the application, purely from that component's own decorator and template. This made AOT fast enough to use even during `ng serve` incremental rebuilds, eliminating JIT's one remaining practical advantage and clearing the way for Angular 9 to make AOT the universal default.

Q11. What observable symptom would tell you an app was still running in JIT mode (say, on an older Angular 8 project)?
Noticeably larger main bundle size (compiler code + raw template strings included), a longer blank-screen interval before first paint versus the same app built with `--prod`/AOT, browser console/network tab showing `reflect-metadata` and `@angular/compiler` in the loaded modules, and, if a strict CSP were applied, a hard failure/warning around `unsafe-eval` that wouldn't occur with an AOT build.

Q12. If asked to migrate a legacy JIT-reliant Angular app to AOT-only, what would you check?
Confirm no component relies on `templateUrl`/decorator metadata being resolved reflectively at runtime in a way the build can't statically see; remove any dynamic-template-from-string usage or explicitly scope/opt it back in with `@angular/compiler` if truly required; enable `strictTemplates` and fix the newly-surfaced build-time template type errors (this is usually where most migration effort goes — bugs JIT was silently hiding); remove `reflect-metadata`/JIT-only polyfills; verify no code path calls `platformBrowserDynamic` or `Compiler.compileModuleSync` unintentionally; and validate bundle size and CSP posture (no `unsafe-eval` needed) post-migration.

7. Quick Revision Cheat Sheet

- **JIT** = compile templates → Ivy instructions **in the browser**, at bootstrap. **AOT** = compile at **build time**; browser only runs pre-compiled JS.
- JIT bootstrap: `platformBrowserDynamic()` + `@angular/compiler` shipped in bundle + `reflect-metadata` needed.
- AOT bootstrap: `platformBrowser()` / `bootstrapModule()` / `bootstrapApplication()` — no compiler shipped, no `unsafe-eval` needed.
- Angular 9 (Ivy) flipped the CLI default from JIT-dev/AOT-prod to **AOT everywhere**, because Ivy's per-component ("local") compilation made AOT fast enough for dev too.
- Why JIT was dropped as default: **bundle bloat** (whole compiler + raw templates shipped), **slower first render** (main-thread parse+codegen before first paint), **errors only at runtime** (no build-time template type-checking), **security** (`eval`-like codegen requires relaxing CSP's `unsafe-eval`).
- `@angular/compiler` = the compiler engine (parsers, AST, emitter). `@angular/compiler-cli` = build-time wrapper (`ngc`/CLI integration) using it — only this runs today, only on the build machine.
- JIT still lives on narrowly: `TestBed` (test-time template overrides, never shipped to users) and an **explicit opt-in escape hatch** (`Compiler.compileModuleSync` / `compileModuleAsync`) for genuinely runtime-unknown templates (CMS/low-code) — using it knowingly reintroduces bundle-size + CSP costs.
- Prefer `NgComponentOutlet` over known, pre-compiled components — or SSR the dynamic content — instead of pulling `@angular/compiler` back into the client bundle.
- Don't confuse Angular's JIT/AOT (template-compilation timing) with V8's JIT (JS-to-machine-code compilation) — unrelated layers.

Created By - Durgesh Singh